VIETNAM NATIONAL UNIVERSITY — HO CHI MINH CITY
UNIVERSITY OF SCIENCE
Faculty of Information Technology

PRACTICAL REPORT 1

COLOR COMPRESSION VIA K-MEANS CLUSTERING

An unsupervised learning approach to RGB palette reduction

Course: **MTH00051 — Applied Mathematics and Statistics**

STUDENT
Nguyễn Thế Hiển
Student ID: 22127107 · Class: 22CLC08

INSTRUCTORS
Mr. Vũ Quốc Hoàng
Mr. Nguyễn Văn Quang Huy
Mr. Nguyễn Ngọc Toàn
Mrs. Phan Thị Phương Uyên

Ho Chi Minh City, 2024

Acknowledgments

This practical exercise forms part of MTH00051 — Applied Mathematics and Statistics, a foundational course for students of Information Technology. Through the Color Compression project, I had the opportunity to connect clustering theory with a concrete computational application.

I gratefully acknowledge the guidance of Mr. Vũ Quốc Hoàng, Mr. Nguyễn Văn Quang Huy, Mr. Nguyễn Ngọc Toàn, and Mrs. Phan Thị Phương Uyên. Their lectures and laboratory support clarified both the mathematical principles and the engineering constraints of the assignment. I also thank my classmates in 22CLC08 for constructive discussion throughout the course.

Any remaining errors in the implementation or in this report are my own.

Abstract

Digital RGB images admit up to 256^3 distinct colors, which makes naive storage expensive. Color compression (color quantization) seeks a compact palette of k representative colors while preserving the visual identity of the scene. In this report, each pixel is modeled as a point in $\mathbb{R}^3$ and clustered by the K-Means algorithm implemented from first principles in Python.

We formalize the optimization objective as minimization of the within-cluster sum of squares, describe Lloyd's alternating minimization procedure, and compare two centroid-initialization schemes required by the course specification: *random* and *in_pixels*. Qualitative experiments for k $\{3, 5, 7\}$ show that larger palettes recover finer tonal structure, whereas initialization mainly affects which local minimum is attained. The study illustrates how a classical unsupervised method links Euclidean geometry, iterative optimization, and practical image processing.

Keywords: color quantization; K-Means clustering; Lloyd iteration; Euclidean distance; image compression; unsupervised learning.

Contents

1. Introduction	4
2. Problem Formulation	4
3. Theoretical Background	5
3.1 K-Means as an Optimization Procedure	5
3.2 Centroid Initialization Strategies	5
4. Implementation	6
4.1 Software Environment	6
4.2 Functional Architecture	6
4.3 Main Program Workflow	7
5. Experimental Results	7
5.1 Experimental Setup	7
5.2 Visual Comparison for k = 3, 5, 7	8
5.3 Qualitative Summary	9
6. Discussion	10
7. Conclusion	10
References	11

List of Figures

Figure 1. Original test image prior to compression.
Figure 2. Reconstructions for k = 3 (in_pixels vs random).
Figure 3. Reconstructions for k = 5 (in_pixels vs random).
Figure 4. Reconstructions for k = 7 (in_pixels vs random).

1. Introduction

1.1. Student Information

Full name	Nguyễn Thế Hiển
Student ID	22127107
Class	22CLC08
Course	MTH00051 — Applied Mathematics and Statistics
Project	Project 01 — Color Compression

1.2. Motivation and Objectives

A color photograph is stored as a rectangular array of pixels. In the RGB model, every pixel carries three channel intensities, each typically encoded by one byte. The resulting discrete color cube $\{0,1,\ldots,255\}^3$ contains up to 256^3 1.68×10^7 admissible colors. Natural images rarely use the entire cube, yet the number of observed colors is still large enough to inflate storage and transmission cost.

Color compression (also called color quantization) replaces the original palette by a much smaller set of prototypes. From the viewpoint of applied statistics, the task is unsupervised clustering in a three-dimensional feature space: similar colors are grouped, and each group is summarized by its centroid. The objectives of this practical report are therefore:

1. to formalize color quantization as an optimization problem on pixel vectors;
2. to implement K-Means from scratch with the two prescribed initializations;
3. to provide an interactive program that exports reconstructed images as png, jpg, or pdf;
4. to evaluate visual quality for several palette sizes and discuss the results rigorously.

2. Problem Formulation

Let an RGB image of height H and width W be represented by the collection of pixel vectors

$$X = \{x_i \in \mathbb{R}^3 \mid i = 1, \ldots, N\}, \quad N = H \cdot W$$

(1)

Given an integer palette size k, one seeks centroids $C = \{c_1, \ldots, c_k\}$ $\mathbb{R}^3$ and an assignment map $\ell : \{1,\ldots,N\}$ $\{1,\ldots,k\}$ that minimize the within-cluster sum of squares (WCSS)

$$J(C, \ell) = \sum_{i=1}^{N} \|x_i - c_{\ell(i)}\|_2^2$$

(2)

The compressed image is then reconstructed by $\hat{x}_i = c_{\ell(i)}$. Consequently, at most k distinct colors appear in the output. Reconstruction fidelity in RGB geometry may be measured by the mean squared error

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N} \|x_i - \hat{x}_i\|_2^2 = \frac{J(C, \ell)}{N}$$

(3)

Equation (3) coincides with the normalized objective (2). Lower MSE indicates a closer Euclidean approximation; perceptual quality, however, also depends on the human visual system and is therefore examined visually in Section 5.

3. Theoretical Background

3.1. K-Means as an Optimization Procedure

Exact global minimization of (2) is NP-hard in general. The classical Lloyd iteration therefore alternates two conditionally optimal steps.

Assignment step. For fixed centroids, every pixel is labeled by the nearest prototype under the squared Euclidean metric:

$$\ell(i) = \arg\min_{j \in \{1, \ldots, k\}} \|x_i - c_j\|_2^2$$

(4)

Update step. For a fixed assignment, each nonempty cluster is summarized by its sample mean—the unique minimizer of the quadratic loss on that subset:

$$c_j \leftarrow \frac{1}{|S_j|}\sum_{i \in S_j} x_i, \quad S_j = \{i \mid \ell(i) = j\}$$

(5)

If a cluster becomes empty, the implementation retains the previous centroid to preserve numerical stability. Iteration stops when centroids change by less than a prescribed absolute tolerance (10^4 in this project) or when a maximum iteration budget is exhausted. Because neither step increases J, the sequence of objective values is monotonically non-increasing and converges to a local minimum.

3.2. Centroid Initialization Strategies

K-Means is sensitive to the initial configuration of centroids. Following the assignment statement, two schemes are implemented.

random. Each centroid coordinate is drawn independently and uniformly from [0, 255]. This explores the full RGB cube and may place prototypes outside the empirical color support of the image, occasionally producing empty clusters in early iterations.

in_pixels. Centroids are sampled uniformly without replacement from observed pixels. The initial palette therefore lies inside the data cloud, which typically yields a more favorable starting value of J and more stable early assignments.

Neither strategy guarantees a global optimum. Comparing both initializations for the same k provides empirical insight into the variability of local solutions.

4. Implementation

4.1. Software Environment

The solution is developed in a Jupyter Notebook and uses only the libraries permitted by the course specification. Pre-built clustering routines (for example, `sklearn.cluster.KMeans`) are excluded from the submission.

Library	Purpose in this project
NumPy	Vectorized distance computation, centroid updates, and array reshaping.
Pillow (PIL)	Image I/O, explicit RGB conversion, and export to png / jpg / pdf.
Matplotlib	Display of original and reconstructed images during experimentation.

4.2. Functional Architecture

The program is decomposed into documented helper functions. The design keeps mathematical operations (clustering) separate from I/O and visualization, which facilitates testing and reuse.

read_img(img_path). Opens an image, converts it to RGB, and returns a floating-point array of shape (H, W, 3).

show_img / save_img. Visualize an image with Matplotlib, or clip intensities to [0, 255] and write an 8-bit file.

convert_img_to_1d(img_2d). Reshapes the image into an (N, 3) design matrix suitable for clustering.

kmeans(...). Executes Lloyd iteration with random or in_pixels initialization; returns centroids and labels.

generate_2d_img(...). Maps every pixel to its centroid color and restores the spatial layout (H, W, 3).

reconstruction_error(...). Computes the MSE in (3), providing a quantitative complement to visual inspection.

4.3. Main Program Workflow

The interactive `main` procedure satisfies the course interface requirements. At each execution the user supplies:

- the path of the input image;
- the number of clusters k and the maximum number of iterations;
- the initialization mode (*random*, *in_pixels*, or both);
- the output format chosen from {png, jpg, pdf};
- a base filename for the reconstructed image(s).

The pipeline then flattens the image, runs K-Means for each requested initialization, displays the reconstruction, reports the MSE, and exports the file in the selected format. Supporting checks are collected in `test_functions`.

5. Experimental Results

5.1. Experimental Setup

Experiments are conducted on a natural beach photograph containing smooth illumination gradients (sky and water) together with a high-contrast silhouette. Such content is informative for quantization: coarse palettes posterize the sky, while larger palettes restore intermediate tones. Both initialization strategies are evaluated for k {3, 5, 7} with a sufficient iteration budget to reach practical convergence.

Figure 1. Original test image prior to color compression.

5.2. Visual Comparison for k = 3, 5, 7

For $k = 3$ (Figure 2), both initializations reduce the scene to a silhouette, a mid-tone band, and a sky tone. Fine water reflections are largely suppressed, yet the semantic layout remains recognizable. The close agreement of the two methods suggests that the dominant chromatic modes are stable under different starts.

Figure 2a. in_pixels initialization, k = 3. *Figure 2b. random initialization, k = 3.*

For k = 5 (Figure 3), additional centroids capture transitional hues near the horizon and richer reflections. Differences between initialization schemes become more noticeable: random sampling may allocate a centroid to a less frequent region of RGB space, slightly altering the tonal balance relative to in_pixels.

Figure 3a. in_pixels initialization, k = 5. *Figure 3b. random initialization, k = 5.*

When k = 7 (Figure 4), reconstructions approach the original more closely. Gradients in the sky and specular highlights on wet sand reappear as distinct palette entries. Residual discrepancies between the two initializations illustrate that K-Means may converge to distinct local minima even for moderate palette sizes.

Figure 4a. in_pixels initialization, k = 7.

Figure 4b. random initialization, k = 7.

5.3. Qualitative Summary

k	Visual fidelity	Effect of initialization
3	Strong posterization; content preserved only at a coarse semantic level.	Minor differences; dominant colors are recovered consistently.
5	Improved mid-tones and horizon detail; water structure becomes clearer.	Noticeable palette shifts between random and in_pixels.
7	Closer to the original; smoother apparent gradients and richer reflections.	Local minima remain visible but are less disruptive perceptually.

6. Discussion

The experiments confirm the classical complexity–fidelity trade-off of prototype methods. Small k yields aggressive compression and a piecewise-constant appearance; larger k reduces distortion and restores chromatic nuance at the expense of a bigger palette. In mathematical terms, the feasible set of reconstructions expands with k, so the minimal attainable value of J is monotonically non-increasing in the palette size.

Concerning initialization, universal claims that one scheme is always “faster” or “better” are not warranted without measuring iteration counts and objective values over repeated trials. The evidence available here supports a more cautious statement: *in_pixels* places centroids on the data manifold and therefore tends to avoid pathological empty-cluster configurations, whereas *random* exploration can discover alternative local minima. For images with concentrated color histograms the two strategies often agree; divergence becomes more likely as k grows.

Several limitations should be acknowledged. First, Euclidean distance in RGB is only an approximate proxy for perceptual dissimilarity; distances in CIELAB would align more closely with human judgment. Second, forming a full distance tensor has memory cost

$O(Nk)$ and may require chunked evaluation for very large images. Third, a single photograph does not exhaust the diversity of natural scenes; a broader benchmark would strengthen statistical conclusions.

7. Conclusion

This practical exercise shows that K-Means clustering provides a clear and effective mechanism for color compression. By casting pixels as points in $\mathbb{R}^3$ and minimizing a quadratic distortion criterion, one obtains a reduced palette that retains the essential visual narrative of an image. The implemented system fulfills the project requirements: a from-scratch K-Means routine with both prescribed initializations, an interactive main program supporting png/jpg/pdf export, and a systematic qualitative evaluation for multiple values of k.

Natural extensions include K-Means++ initialization, multi-run selection of the best local minimum, and quantitative reporting of MSE/PSNR across a curated image set. Such enhancements would further connect the algorithmic prototype developed here with contemporary practice in statistical image analysis.

References

[1] MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, 1, 281–297.

[2] Lloyd, S. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2), 129–137.

[3] Arthur, D., & Vassilvitskii, S. (2007). k-means++: The advantages of careful seeding. *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1027–1035.

[4] Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer. (Chapter 9: Mixture models and EM; background on prototype-based clustering).

[5] Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362. https://numpy.org/doc/

[6] Clark, A., and contributors. (n.d.). *Pillow (PIL Fork) Documentation*. https://pillow.readthedocs.io/en/stable/

[7] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. https://matplotlib.org/stable/